Documento Técnico: TacticalGrid

Integrantes: Cristian David Gonzalez - Sebastian Garcia Hincapie

Docente

Jairo Iván Vélez Bedoya

Sistemas Inteligentes I, Proyecto 1. Universidad de Caldas.

2026-2

Tabla de contenido

Documento Técnico: TacticalGrid 1
1. Descripción del problema 3
2. Representación del escenario 3
3. Unidades 4
4. Estado de navegación 4
5. Estado de juego 6
6. Acciones 7
7. Costo de las acciones 8
8. Condición objetivo y estados terminales 9
9. Reglas deterministas 10
10. Formulación adversarial 11
11. Ejemplo trazado 13
12. Heurísticas 14
13. Arquitectura del software 16
14. Carga y validación de escenarios 18
15. Verificación de soluciones 20
16. Metodología experimental 21
17. Resultados y análisis 24
18. Análisis de complejidad 30
19. Supuestos y limitaciones 31
20. Reporte de uso de IA 32

1. Descripción del problema

Es un juego por turnos sobre una cuadrícula vista desde arriba. Dos bandos, A y B, controlan cada uno al menos tres unidades y tienen una base propia. En una zona neutral hay un recurso estratégico: una unidad lo recoge al llegar a su casilla y su bando gana si lo transporta hasta su propia base. El mapa combina terrenos con distinto costo de paso y obstáculos que no se pueden atravesar.

El modelo se usa de dos formas. En navegación, una sola unidad busca un camino desde su posición hasta un objetivo mientras las demás unidades permanecen inmóviles, esta forma la resuelven BFS, DFS, UCS, A* y Beam Search. En el juego adversarial, ambos bandos actúan por turnos y cada decisión tiene en cuenta las respuestas posibles del rival; esta forma la resuelven Minimax y Minimax con poda alfa-beta.

Cada forma tiene su propio estado. El estado de navegación (sección 4) describe solo a la unidad que se mueve, el estado de juego (sección 5) describe la partida completa.

2. Representación del escenario

El escenario es la información que no cambia durante una búsqueda ni durante una partida. Se construye una sola vez a partir del JSON validado, y los algoritmos solo lo consultan.

Contiene las dimensiones del mapa (mapa.filas y mapa.columnas), el catálogo de terrenos (tipos_terreno, con el costo y la transitabilidad de cada tipo), la matriz de terreno (terreno[f][c] es el tipo de la celda en la fila f y la columna c) y las posiciones de las bases A y B.

Convención de coordenadas: una celda se identifica con el par (fila, columna), con origen (0, 0) en la esquina superior izquierda; la fila crece hacia abajo y la columna hacia la derecha, igual que en el JSON. Una celda es válida si $0 \leq$ fila $<$ filas y $0 \leq$ columna $<$ columnas.

Los costos y la transitabilidad se leen siempre del catálogo de terrenos. Ningún algoritmo tiene costos escritos en el código, así que cambiar un costo en el JSON (por ejemplo, el pantano de 7 a 15) cambia el comportamiento de UCS y A* sin modificar el programa.

Las posiciones del recurso y de las unidades, el portador y el turno no forman parte del escenario: cambian durante la partida y por eso pertenecen al estado (secciones 4 y 5).

3. Unidades

Todas las unidades son idénticas (tipo estándar). Comparten las mismas acciones, los mismos costos de terreno y las mismas reglas sin importar a qué bando pertenecen.

El enunciado permite unidades idénticas mientras el problema siga siendo claro. Con un solo tipo hay menos reglas que programar y verificar el factor de ramificación de Minimax depende solo del número de unidades y de acciones, y la simetría entre bandos es directa: MAX y MIN disponen exactamente de las mismas capacidades lo que sostiene el argumento de que MIN no está en desventaja.

Cada unidad se describe con su identificador (id, único), su bando (A o B) y su posición. El campo tipo del JSON se lee y se conserva para mostrarlo en la interfaz, pero no cambia el comportamiento: si un escenario externo tiene un valor distinto de estándar, la unidad se trata igual que una estándar. Así cualquier escenario válido se puede cargar sin modificar el código ya que la validación exigida por el enunciado no restringe los valores de tipo.

4. Estado de navegación

Regla sobre las demás unidades: En navegación solo cuenta el terreno. Las demás unidades, aliadas o rivales, no son obstáculos y no intervienen en la generación de sucesores ni en el cálculo de costos.

El enunciado indica que en la etapa de búsqueda se ignora temporalmente el comportamiento del adversario y el propio escenario de ejemplo pide llevar A1 hasta la casilla (3, 2), ocupada por B1; si las unidades bloquearan ese caso no tendría solución. Además, así el resultado de una búsqueda depende solo del mapa y de los costos del JSON. Esta regla aplica únicamente a BFS, DFS, UCS, A* y Beam Search. La ocupación de casillas durante la partida es una regla distinta definida en la sección 9 y las distancias que use la función de utilidad (sección 10) se calculan

con la regla de navegación.

El estado de navegación es la tupla (fila, columna, lleva_recurso). fila y columna son la posición de la unidad que se mueve; lleva_recurso indica si la unidad transporta el recurso.

Estado inicial: La posición de prueba.unidad_inicio. lleva_recurso es verdadero si esa unidad es juego.portador_recurso y falso en cualquier otro caso.

Sucesores: Desde (f, c, r) se genera un sucesor por cada celda vecina (arriba, abajo, izquierda, derecha) que esté dentro del mapa y sea transitable según el catálogo de terrenos. Si la celda destino es la posición del recurso y r es falso, el sucesor tiene lleva_recurso verdadero; en cualquier otro caso conserva el valor de r. Una vez verdadero, el indicador no vuelve a falso durante la búsqueda, y si el recurso lo transporta otra unidad no hay casilla de recurso, así que el indicador no cambia. El orden de generación de los vecinos se fija en la sección 6.

Objetivo: En una prueba de navegación normal llegar a la celda prueba.objetivo, sin importar el valor de lleva_recurso. En la misión completa, solicitada con el campo propio prueba.mision con valor llevar_recurso, el objetivo es estar en la base del bando de la unidad con lleva_recurso verdadero.

Suficiencia: Los sucesores dependen solo de la posición y del terreno, que es fijo; el costo depende solo de la celda destino y el objetivo depende de la posición y en la misión completa de lleva_recurso. Ningún otro dato influye en estas tres respuestas por lo que la tupla contiene la información necesaria y ninguna sobrante.

Por qué la posición no basta: En el escenario de ejemplo, A1 parte de (0, 1), el recurso está en (2, 2) y la base A en (0, 0). La misión completa baja por la columna 0 hasta el recurso y vuelve a subir por las mismas casillas. Con solo (fila, columna), la casilla (1, 0) quedaría marcada como visitada a la ida y la búsqueda descartaría el camino de vuelta. Con el indicador, (1, 0, falso) y (1, 0, verdadero) son estados distintos y la solución existe: (0,1) → (0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (2,1) → (2,0) → (1,0) → (0,0)con costo 15. Este es el caso mínimo 6: la misma posición física representa estados diferentes.

Tamaño del espacio de estados: como máximo dos estados por celda transitable. En un mapa de 20 × 20 son a lo sumo 800 estados, un tamaño que no afecta el rendimiento de ningún algoritmo.

5. Estado de juego

Ninguna unidad se elimina durante la partida. Interceptar al portador sólo afecta al recurso (regla definida en la sección 9) todas las unidades permanecen en el tablero hasta el final. Así cada bando conserva siempre el mismo número de unidades, se mantiene la simetría entre MAX y MIN y se evita un sistema de combate, como pide el enunciado.

Campos del estado de juego:

- **Posiciones:** La posición (fila, columna) de cada unidad, identificada por su id. Se guarda ordenada por id para que dos estados con las mismas posiciones se representan siempre igual.
- **Recurso:** La posición del recurso cuando está en el suelo o vacío cuando alguna unidad lo transporta.
- **Portador:** El id de la unidad que transporta el recurso o vacío si nadie lo lleva. Siempre se cumple que exactamente uno de los dos campos, recurso o portador, tiene valor cuando hay portador, el recurso está en la posición de esa unidad.
- **Turno:** El bando que debe actuar, A o B.
- **turnos_jugados:** El número de turnos transcurridos desde el inicio de la partida, necesario para aplicar el límite de turnos y declarar empate (el valor del límite se fija en la sección 8).

El bando de cada unidad no se guarda en el estado porque nunca cambia: se toma del escenario a partir del id.

Estado inicial: Se construye con los campos unidades, recurso, turno y juego.portador_recurso del JSON, con turnos_jugados en 0. Si portador_recurso no es null, el recurso se considera en poder de esa unidad y la posición del campo recurso del JSON no se usa.

Suficiencia: Con estos campos y el escenario se puede decidir qué acciones son legales para el

bando que tiene el turno (posiciones y terreno), qué estado produce cada acción (posiciones, recurso y portador), si la partida terminó y quién ganó (posición del portador frente a su base y turnos_jugados), y calcular la función de utilidad (posiciones, recurso, portador y turno). Ningún otro dato interviene en esas decisiones.

Estado compuesto: Dos estados con A1 en la misma casilla son distintos si en uno A1 es el portador y en el otro el recurso está en el suelo. En el primero el objetivo de A1 es su base y el rival intentará interceptarlo; en el segundo, A1 debe ir por el recurso. Este es el caso mínimo 6 en el juego.

Implementación: El estado es inmutable (cada acción produce un estado nuevo y nunca modifica el anterior) y comparable por valor (dos estados con los mismos datos son iguales y tienen el mismo hash). Esto permite que Minimax explore ramas sin efectos secundarios y que los estados se usen en conjuntos y diccionarios.

6. Acciones

En cada turno el bando que tiene el turno elige una sola jugada. Una jugada consiste en mover una de sus unidades de una casilla en una de las cuatro direcciones, o en pasar.

Jugadas posibles:

- Mover (unidad, dirección) con dirección arriba, derecha, abajo o izquierda.
    - **Precondición:** La unidad pertenece al bando que tiene el turno y la celda destino está dentro del mapa, es transitable y cumple las reglas de ocupación y de bases de la sección 9.
    - **Efecto:** La unidad pasa a la celda destino y luego se aplican los efectos automáticos.
- Pasar.
    - **Precondición:** Ninguna, siempre es legal.
    - **Efecto:** Ninguna unidad se mueve. Se cuenta como una sola jugada por turno, porque que se quede quieta cualquiera de las unidades produce el mismo estado.

Después de cualquier jugada, el turno pasa al otro bando y turnos_jugados aumenta en 1.

Efectos automáticos de un movimiento, que no son jugadas aparte: Si la unidad entra a la casilla del recurso y nadie lo transporta, pasa a ser la portadora; si la unidad alcanza al portador rival, se aplica la regla de intercepción de la sección 9; y si el portador entra a la base de su bando, la partida termina con su victoria.

Las acciones que menciona el enunciado (avanzar, aproximarse al recurso, bloquear una posición, proteger una zona, interceptar al portador) surgen de combinar movimientos y la jugada pasar, sin reglas adicionales. Bloquear es ocupar una casilla de paso; proteger es mantenerse cerca de una zona; interceptar es moverse hacia el portador. Con u unidades por bando hay como máximo 4u + 1 jugadas por turno (13 con tres unidades), lo que mantiene manejable el árbol de Minimax. La jugada pasar garantiza que siempre exista al menos una jugada legal y permite mantener un bloqueo sin abandonarlo.

Orden de generación: Por defecto, las unidades se recorren por id en orden ascendente; para cada unidad, las direcciones se generan en el orden arriba, derecha, abajo, izquierda, y la jugada pasar va al final. En navegación los vecinos se generan con el mismo orden de direcciones. El orden es un parámetro configurable sin modificar el código, porque el enunciado pide cambiarlo y analizar su efecto en DFS y en la poda alfa-beta.

7. Costo de las acciones

Mover una unidad de una celda a una vecina cuesta el costo del terreno de la celda destino, tomado del catálogo tipos_terreno del JSON. La celda inicial no se cobra, porque la unidad ya está en ella.

El costo de un camino es la suma de los costos de todos sus movimientos. En el escenario de ejemplo, el camino de BFS hasta (3, 2) pasa por pasto, bosque, pantano y camino, con costo 2 + 4 + 7 + 1 = 14; el de UCS rodea por la columna 0 y cuesta 1 + 1 + 1 + 1 + 2 + 1 = 7. Esta diferencia es la que muestra el caso mínimo 2.

La jugada pasar que solo existe en la partida, tiene costo 0 porque no se atraviesa ningún terreno.

Cobrar el destino corresponde a la idea de pagar por el terreno que se pisa y es la convención más usada en búsqueda sobre cuadrículas. La validación garantiza que todo terreno transitable tiene costo positivo, condición que necesitan UCS y A* para encontrar el camino de menor costo. Como los costos se leen del JSON, cambiar el costo de un terreno cambia los resultados de UCS y A* sin modificar el código y el verificador de soluciones recalcula esta misma suma para comprobar el costo que reporta cada algoritmo.

En la partida, Minimax decide con la función de utilidad (sección 10) y no con el costo acumulado; el costo de terreno interviene allí solo a través de las distancias calculadas con esta misma convención.

8. Condición objetivo y estados terminales

Navegación: En una prueba normal, el objetivo es que la unidad esté en la celda prueba.objetivo. En la misión completa, el objetivo es que la unidad esté en la base de su bando con lleva_recurso verdadero (sección 4).

Partida, victoria: Un bando gana en el momento en que su portador entra a la base de su propio bando. La condición se revisa inmediatamente después de cada jugada. Ninguna unidad, incluido el portador, puede entrar a la base rival (regla 2 de la sección 9). Si un escenario cargado ya empieza con el portador sobre su propia base, la partida se considera terminada con la victoria de ese bando.

Partida, empate: Si turnos_jugados alcanza el límite de turnos sin que ningún bando haya ganado, la partida termina en empate.

El límite de turnos es configurable. Se lee del campo propio opcional juego.limite_turnos del JSON; si el escenario no trae ese campo, se usa el valor por defecto de 200 turnos (100 por bando). Como el campo es opcional, cualquier escenario externo que respete el formato común se carga sin cambios.

Sin límite, una partida en la que ambos bandos se bloquean mutuamente podría no terminar. El valor de 200 deja margen suficiente en mapas de 20 × 20, donde cruzar el mapa puede requerir unos 40 movimientos y en cada turno solo se mueve una unidad. Que sea configurable permite preparar escenarios de prueba con un límite pequeño para demostrar el empate.

Estados terminales para Minimax: Un estado es terminal si hay ganador o si se alcanzó el límite de turnos. En esos estados la utilidad se calcula directamente (sección 10). Alcanzar la profundidad máxima de búsqueda no hace terminal a un estado: en ese caso se aplica la función de evaluación.

9. Reglas deterministas

Las reglas de esta sección aplican a la partida (modos partida y adversarial). La navegación usa únicamente las reglas de la sección 4.

Regla 1, ocupación: Una casilla admite como máximo una unidad. Moverse a una casilla ocupada por cualquier unidad, aliada o rival, no es legal.

- Esta regla es la que hace posible bloquear una posición, una de las acciones que menciona el enunciado; una unidad detenida en un paso estrecho cierra el camino del rival.

Regla 2, bases: Ninguna unidad puede entrar a la base del bando rival, cada bando sólo puede pisar su propia base.

- Como las unidades bloquean, una unidad estacionada sobre la base rival impediría para siempre que el portador rival entrara y ese bando no podría ganar. Con esta regla la base se defiende rodeándola e interceptando, pero nunca se tapa.

Regla 3, recoger el recurso: Una unidad recoge el recurso cuando entra a su casilla y nadie lo transporta. Si un escenario coloca una unidad sobre el recurso desde el inicio, esa unidad no lo recoge hasta salir de la casilla y volver a entrar.

Regla 4, intercepción: Una unidad intercepta al portador rival cuando se mueve a una casilla

vecina de él (arriba, abajo, izquierda o derecha; la diagonal no cuenta). El recurso pasa de inmediato a la unidad que interceptó. Solo cuenta el movimiento de quien intercepta: si es el portador quien queda vecino de una unidad rival, no ocurre nada.

- **Consecuencia:** La unidad que pierde el recurso no puede recuperarlo en el turno siguiente, porque desde una casilla vecina cualquier movimiento legal la deja a dos casillas del nuevo portador, otra unidad de su bando sí puede interceptar, y ella misma puede hacerlo en turnos posteriores. Esto evita robos alternados sin fin entre dos unidades.

Regla 5, unidad o bando sin movimientos: Si una unidad no tiene movimientos legales, simplemente no genera jugadas. Como la jugada pasar siempre es legal, un bando nunca queda sin jugadas.

Regla 6, ausencia de ruta en navegación: Si no existe camino hasta el objetivo, la búsqueda termina con exito igual a falso, sin error, y se reportan sus métricas.

Regla 7, límite de turnos: Al alcanzar el límite definido en la sección 8 sin ganador, la partida termina en empate.

Regla 8, escenarios iniciales: Un escenario válido se acepta tal como viene, aunque tenga dos unidades en la misma casilla o un portador ya vecino de una unidad rival. Las reglas se aplican a partir de la primera jugada.

Orden de aplicación: Después de cada jugada se aplica, en este orden, la recogida del recurso, la intercepción, el aumento de turnos_jugados con el cambio de turno, la revisión de victoria y, si no hay ganador, la revisión del límite de turnos. Si la unidad que intercepta está sobre su propia base, gana en ese mismo momento.

10. Formulación adversarial

Decisión sobre los roles: MAX es el bando que tiene el turno en el estado raíz de cada búsqueda y MIN es el otro bando. Dentro de una búsqueda MAX no cambia, y los niveles del árbol

alternan entre jugadas de MAX y de MIN. Cada decisión es una búsqueda nueva: en el modo adversarial, MAX lo define el campo turno del JSON; en una partida entre dos IA, cuando juega A se busca con MAX igual a A y cuando juega B, con MAX igual a B.

Por qué MIN no está en desventaja: MAX y MIN son roles, no bandos con capacidades distintas. Ambos tienen las mismas reglas, las mismas jugadas y la misma profundidad, y la utilidad se calcula con la misma fórmula para los dos. El juego es de suma cero: lo que mejora la situación de un bando empeora en la misma medida la del otro, así que minimizar la utilidad de MAX equivale a maximizar la propia.

Utilidad de los estados terminales: +10 000 si gana MAX, −10 000 si gana MIN y 0 si hay empate.

Evaluación de los estados no terminales: Cuando la búsqueda llega a la profundidad máxima, el estado se evalúa como utilidad = puntaje(MAX) − puntaje(MIN). El puntaje de un bando suma los siguientes factores:

- Posesión: +100 si el bando lleva el recurso.
- Avance del portador: −5 por cada unidad de costo entre el portador del bando y su base, si el bando lleva el recurso.
- Carrera por el recurso: −2 por cada unidad de costo entre la unidad del bando más cercana al recurso y el recurso, si está en el suelo.
- Persecución: −3 por cada unidad de costo entre la unidad del bando más cercana al portador rival y ese portador, si el rival lleva el recurso.

Distancias: Todas las distancias son el costo mínimo de navegación según el terreno (secciones 4 y 7), sin considerar unidades. Al cargar el escenario se precalculan las distancias entre todas las celdas transitables con una ejecución de Dijkstra desde cada celda (unas 400 ejecuciones en un mapa de 20 × 20), de modo que evaluar un estado solo requiere consultar una tabla. Si un destino es inalcanzable, se usa un valor grande fijo.

Pesos: los pesos (100, 5, 2 y 3) se leen de la configuración y no están escritos en la lógica, para

poder modificarlos y analizar el cambio de comportamiento. Son valores iniciales; si las partidas de prueba muestran un comportamiento indeseado, se ajustarán y el cambio se registrará en este documento.

La función es simétrica, porque calcula lo mismo para ambos bandos y resta, lo que refleja que el juego es de suma cero. Interceptar al portador rival cambia la utilidad en unos 200 puntos, de modo que Minimax puede preferir interceptar antes que avanzar hacia el recurso, que es lo que muestra el caso mínimo 7.

Profundidad y desempates: La profundidad máxima se lee de juego.profundidad_maxima_minimax o de la interfaz, sin cambiar el código. Si dos jugadas tienen el mismo valor, se elige la primera según el orden de generación de la sección 6. Como Minimax y alfa-beta usan la misma utilidad, el mismo orden y la misma regla de desempate, con la misma profundidad seleccionan la misma jugada.

11. Ejemplo trazado

Escenario usado: el JSON de ejemplo del enunciado, de 4 × 4. Base A en (0, 0), base B en (3, 3), recurso en (2, 2), muro en (1, 1); unidades A1 en (0, 1), A2 en (1, 0), B1 en (3, 2) y B2 en (2, 3); turno de A y ningún portador. El ejemplo tiene dos unidades por bando; los escenarios del equipo tendrán al menos tres.

Navegación. La prueba del JSON pide llevar A1 hasta (3, 2). El estado inicial es (0, 1, falso). Con el orden de la sección 6, sus vecinos son: arriba, fuera del mapa; derecha, (0, 2) de pasto; abajo, (1, 1) que es muro; izquierda, (0, 0) de camino. Por tanto los sucesores son (0, 2, falso) con costo 2 y (0, 0, falso) con costo 1, en ese orden. BFS encuentra el camino de 4 movimientos y costo 14, y UCS el de 6 movimientos y costo 7 (sección 7). En la misión completa, la solución es el camino de ida y vuelta de costo 15 descrito en la sección 4.

Partida, estado inicial: posiciones A1 (0, 1), A2 (1, 0), B1 (3, 2), B2 (2, 3); recurso en (2, 2); portador vacío; turno A; turnos_jugados 0.

Jugadas legales de A, en el orden de generación: A1 a la derecha, a (0, 2); A1 a la izquierda, a (0,

0), que es su propia base; A2 arriba, a (0, 0); A2 abajo, a (2, 0); y pasar. Se descartan A1 arriba (fuera del mapa), A1 abajo y A2 a la derecha (muro) y A2 a la izquierda (fuera del mapa). Son 5 jugadas.

Secuencia de ejemplo de tres jugadas:

1. Turno 1, A: A2 baja a (2, 0). No recoge ni intercepta nada; el turno pasa a B.
2. Turno 2, B: B1 sube a (2, 2), la casilla del recurso, y lo recoge. B1 es ahora el portador.
3. Turno 3, A: A2 se mueve a la derecha, a (2, 1). Queda vecino de B1, así que lo intercepta y el recurso pasa a A2.

Utilidad con MAX igual a A, antes y después del turno 3. Antes: la unidad de A más cercana al portador B1 es A2, a costo 8, así que puntaje(A) = −3 × 8 = −24; B1 lleva el recurso a costo 2 de su base, así que puntaje(B) = 100 − 5 × 2 = 90; la utilidad es −24 − 90 = −114. Después: A2 lleva el recurso a costo 3 de la base A, así que puntaje(A) = 100 − 5 × 3 = 85; la unidad de B más cercana a A2 es B1, a costo 1, así que puntaje(B) = −3 × 1 = −3; la utilidad es 85 − (−3) = 88. La intercepción cambia la utilidad en 202 puntos, por eso Minimax la prefiere frente a otras jugadas.

Estado compuesto: Después del turno 3, A2 está en (2, 1) llevando el recurso. Si en el turno 2 B hubiera pasado en lugar de recoger el recurso, A2 podría estar en la misma casilla (2, 1) con el recurso en el suelo. Son estados distintos: en el primero A2 debe ir a su base y B intentará interceptarlo; en el segundo A2 debe ir por el recurso.

12. Heurísticas

Las heurísticas se usan solo en el modo búsqueda: A* ordena la frontera por f(n) = g(n) + h(n) y Beam Search conserva en cada nivel los k estados con menor f(n) = g(n) + h(n).
BFS, DFS y UCS no usan heurística, y Minimax usa la función de utilidad de la sección 10, que es otro concepto.
Costo mínimo: c_min es el menor costo entre los terrenos transitables del catálogo tipos_terreno. Ambas heurísticas multiplican por c_min para expresar la estimación en unidades

de costo y no sobreestimar aunque el JSON cambie los costos.

Heurística 1, Manhattan: h1(n) = (|fila − fila_objetivo| + |columna − columna_objetivo|) × c_min. En el escenario de ejemplo c_min = 1, así que coincide con la distancia Manhattan clásica. Es admisible porque ningún camino tiene menos pasos que la distancia Manhattan y cada paso cuesta al menos c_min. Se calcula en O(1).

Heurística 2, pasos reales rodeando obstáculos: h2(n) = pasos(n) × c_min, donde pasos(n) es el menor número de movimientos entre n y el objetivo esquivando los muros, sin tener en cuenta los costos del terreno. pasos se calcula con un BFS desde el objetivo al comenzar la búsqueda. Respuestas a lo que pide el enunciado sobre la heurística 2:

- Qué estima: el costo mínimo que falta para llegar al objetivo, teniendo en cuenta que los obstáculos obligan a rodear.
- Qué información del estado usa: la posición de la unidad (y lleva_recurso en la misión completa), junto con el mapa de obstáculos y c_min del escenario.
- Costo computacional: un BFS desde el objetivo, O(N) en tiempo y en memoria con N celdas, una sola vez por búsqueda, después, cada consulta es O(1).
- Admisibilidad: el camino real tiene al menos pasos(n) movimientos, porque el BFS da el mínimo de pasos esquivando muros y cada movimiento cuesta al menos c_min; por tanto, h2(n) nunca supera el costo real. También es consistente: un movimiento cambia pasos(n) en a lo sumo 1 y cuesta al menos c_min.
- Dónde orienta bien: en mapas con muros y pasillos, donde Manhattan subestima mucho porque no ve los obstáculos.
- Dónde pierde discriminación: en mapas sin muros coincide con Manhattan y no aporta información adicional y cuando los terrenos tienen costos muy distintos, porque cuenta pasos y no costos, así que no distingue una ruta por pantano de otra de igual longitud por camino.

Relación entre las dos: pasos(n) nunca es menor que la distancia Manhattan, así que h2(n) ≥ h1(n) en todo estado; h2 domina a h1. Como ambas son admisibles y consistentes, A* encuentra el camino de menor costo con cualquiera de las dos, y con h2 no expande más estados que con h1, salvo diferencias por empates.

Ejemplo: en un mapa de 5 × 7 con un muro vertical entre la salida y la meta y todo el terreno de costo 1, A* encuentra con ambas heurísticas el camino óptimo de costo 10, pero con Manhattan expande 24 estados y con h2 expande 10 (desempatando por menor h). Manhattan considera prometedoras las casillas pegadas al muro y explora el bolsillo frente a él antes de rodearlo; h2 sabe que desde esas casillas faltan varios pasos y rodea directamente. Este escenario corresponde a los casos mínimos 3 y 4.

Misión completa: Si la unidad no lleva el recurso, la heurística es la estimación desde su posición hasta el recurso más la estimación desde el recurso hasta su base; si ya lo lleva, es la estimación hasta su base. Para h2 se calculan dos BFS, uno desde el recurso y otro desde la base. Sigue siendo admisible, porque todo camino de la misión pasa por el recurso.

Criterio de Beam Search: Beam Search se implementa como A* con la frontera limitada a k estados. Se expande el estado de menor f(n) = g(n) + h(n), usando la heurística elegida (h1 o h2), y después de agregar sus sucesores, si la frontera tiene más de k estados se conservan solo los k de menor f y el resto se descarta definitivamente. Los empates se resuelven por menor h y luego por orden de generación. Ordenar solo por h ignoraría lo ya gastado y favorecería caminos que atraviesan terrenos caros pero quedan cerca de la meta; con f, el criterio tiene en cuenta el costo que es el centro del problema. Con k pequeño Beam puede descartar el camino óptimo y devolver uno más caro o ninguno al aumentar k se acerca a A* y con k suficientemente grande coincide con él lo que permite compararlo directamente con A*.

13. Arquitectura del software

El enunciado exige separar la lógica del juego, la representación del escenario, los algoritmos de búsqueda y decisión, y la interfaz gráfica. El código se organiza en un paquete de Python llamado tacticalgrid, dividido en seis capas. Los datos y las herramientas de trabajo viven fuera de él.

core: Modelo del problema. Contiene el escenario (información fija del mapa), el estado de navegación, el estado de juego, las jugadas y las reglas deterministas (secciones 2 a 9). No depende de ninguna otra capa.

carga: Lectura del archivo JSON, validación de sus reglas y construcción del escenario y del

estado inicial. La carga es completa o no se aplica (sección 14). Depende solo de core.
busqueda: Interfaz común de problema (estado inicial, sucesores, objetivo y costo), los algoritmos BFS, DFS, UCS, A* y Beam Search, las heurísticas (sección 12), el cálculo previo de distancias y el registro de métricas. Depende solo de core.
adversarial: Función de utilidad, Minimax, Minimax con poda alfa-beta y estrategias de orden de jugadas (sección 10). Depende de core y usa las distancias precalculadas de busqueda.
verificacion: Comprobación independiente de caminos y jugadas (sección 15). Depende solo de core y no reutiliza los algoritmos que verifica, para no validar un resultado con el mismo código que lo produjo.
ui: Interfaz gráfica en Tkinter. Dibuja el mapa, permite cargar escenarios, elegir el modo, el algoritmo y sus parámetros, y muestra las métricas. Es la única capa que puede usar a todas las demás.
Regla de dependencias: Las dependencias van en un solo sentido, de ui hacia core. Ninguna capa de lógica importa ui, y los algoritmos solo conocen la interfaz de problema y el escenario, nunca un mapa concreto ni posiciones escritas en el código. Así la lógica se prueba sin abrir la interfaz y cualquier escenario externo se usa sin modificar el programa.
Configuración: Los valores por defecto (límite de turnos, pesos de la función de utilidad y orden de las direcciones) se reúnen en un módulo de configuración del paquete, en lugar de quedar repartidos dentro de los algoritmos.
Flujo de una ejecución: archivo JSON → carga (leer, validar y construir) → escenario y estado inicial → algoritmo de búsqueda o de decisión → resultado con métricas → verificación → interfaz.
Fuera del paquete: Scenarios guarda los archivos JSON (dev para desarrollo, casos para los 10 casos mínimos e invalidos para probar la validación), tests contiene las pruebas automáticas con pytest, experiments, los scripts que ejecutan los casos y sus resultados y docs, la documentación.
Herramientas: Python 3.11 o superior y Tkinter, que viene incluido con Python. El proyecto se describe en pyproject.toml: el programa no tiene dependencias externas, y el grupo opcional dev reúne pytest, ruff, pandas y matplotlib. Se instala con pip install -e ".[dev]".

14. Carga y validación de escenarios

Todo escenario entra al sistema por una sola función, cargar_escenario(ruta), que ejecuta tres pasos en orden: leer el archivo (leer_json), validar su contenido (validar) y construir los objetos del modelo (construir_datos). El resultado es un DatosEscenario listo para usar o una excepción; nunca un escenario a medias.

Lectura del archivo: leer_json lanza ErrorCarga con un mensaje claro cuando el archivo no existe, cuando la ruta es una carpeta, cuando el contenido no está en UTF-8 (se acepta UTF-8 con BOM, frecuente en archivos guardados desde Windows), cuando el JSON está mal formado (el mensaje indica la línea y la columna del error) y cuando la raíz del documento no es un objeto.

Validación: validar(datos) recibe el contenido leído y devuelve la lista de todos los errores encontrados; una lista vacía significa que el escenario es válido. No lanza excepciones aunque falten campos o tengan tipos equivocados. Reporta todos los errores a la vez, en un orden fijo (mapa, tipos de terreno, terreno, bases, unidades, juego y prueba), y evita errores en cascada: si una parte de la que dependen otras reglas es inválida (por ejemplo, las dimensiones del mapa), esas reglas dependientes no se revisan, para no llenar el reporte de errores que son consecuencia del primero. Cada mensaje indica la ruta del campo con problema, por ejemplo unidades[2].posicion.

Reglas del enunciado: R1. filas y columnas son enteros positivos y terreno tiene exactamente esas dimensiones. R2. Cada nombre de terreno de la matriz existe en tipos_terreno. R3. Cada tipo de terreno tiene transitable booleano y los transitables tienen costo numérico positivo. R4. Bases, recurso y unidades están dentro del mapa. R5. Ninguna unidad está sobre un terreno no transitable. R6. No hay ids de unidad repetidos. R7. Cada unidad tiene bando A o B. R8. El turno es A o B. R9. Si juego.portador_recurso no es null, es el id de una unidad existente.

Reglas propias del equipo: Además de las nueve reglas, el validador exige: que exista al menos un terreno transitable, que las bases y el recurso estén sobre celdas transitables, que los costos sean números finitos (no infinito ni NaN), que los enteros no sean booleanos (en JSON true no cuenta como 1), que juego.profundidad_maxima_minimax y juego.limite_turnos, si aparecen,

sean enteros positivos, que prueba.modo sea busqueda, partida o adversarial y en modo busqueda, que unidad_inicio sea una unidad existente y que la prueba traiga un objetivo dentro del mapa o la misión llevar_recurso.

Mayúsculas y minúsculas: Los bandos de las unidades, las claves de las bases y el turno se aceptan en mayúscula o minúscula y se normalizan a mayúscula al construir los objetos, de modo que el resto del sistema solo trabaja con A y B.

Carga completa o nula: Si validar devuelve errores, cargar_escenario lanza ErrorValidacion (una subclase de ErrorCarga) con la lista completa de errores y un mensaje resumen del tipo «El escenario no es válido: N error(es)», seguido de un error por línea. No se construye ningún objeto. La interfaz mostrará ese mensaje y conservará el escenario que estaba cargado antes.

Campos propios del formato: Todos son opcionales, de modo que los archivos del docente cargan sin cambios. juego.limite_turnos fija el máximo de turnos de una partida si no viene se usa 200 (config.LIMITE_TURNOS_POR_DEFECTO) y un null explícito se rechaza. prueba.mision con valor llevar_recurso indica que la búsqueda debe recoger el recurso y llevarlo a la base propia, en lugar de llegar a una celda objetivo. juego.profundidad_maxima_minimax puede faltar o ser null y en ese caso queda sin valor. El campo tipo de cada unidad se conserva tal como viene (en este proyecto todas son estandar).

Construcción: construir_datos convierte el contenido validado en objetos inmutables: Escenario (dimensiones, catálogo de terrenos, matriz de terreno y bases), la lista de UnidadInicial, la posición del recurso, el turno, el portador, la profundidad, el límite de turnos y la Prueba si existe. Son dataclasses congeladas y los diccionarios internos son de solo lectura, así que ningún algoritmo puede modificar el escenario por accidente.

Pruebas: tests/test_cargador.py (20 pruebas) verifica la carga del ejemplo 4×4 del enunciado, los valores por defecto, la normalización a mayúscula, los errores de lectura y la inmutabilidad. tests/test_validador.py verifica que el ejemplo no tiene errores, que cada archivo de scenarios/invalidos (r1_dimensiones.json a r9_portador_inexistente.json, uno por regla) produce exactamente un error, que varios errores se reportan juntos, que valores basura o campos borrados en cualquier parte no producen excepciones, y las reglas propias del equipo. Todas las pruebas pasan.

15. Verificación de soluciones

El verificador (módulo verificacion) revisa la respuesta de un algoritmo sin confiar en él: recibe el camino o la partida ya calculados y comprueba con los datos del escenario que la respuesta sea válida y que sus valores sean reales. Devuelve un reporte con la lista completa de errores encontrados, cada uno indicando el paso o turno donde ocurre, el reporte es válido solo si la lista está vacía. No se detiene en el primer error para que una respuesta con varios defectos los muestre todos.

Caminos de búsqueda: verificar_camino recibe el escenario y el resultado de cualquiera de los cinco algoritmos (BFS, DFS, UCS, A* o Beam Search). Es independiente de los algoritmos y del problema de navegación: no los importa, y vuelve a aplicar las reglas de la sección 4 usando únicamente el mapa, los costos, la posición de la unidad, el recurso y el objetivo del JSON. Comprueba:

1. Inicio: El primer estado es la posición de prueba.unidad_inicio, con lleva_recurso verdadero solo si esa unidad es el portador.

2. Cada paso: La acción es una de las cuatro direcciones, el estado siguiente es exactamente el anterior desplazado una casilla en esa dirección, el destino está dentro del mapa y es transitable y el número de estados es el número de acciones más uno.

3. Recurso: lleva_recurso solo se activa al entrar a la casilla del recurso cuando nadie lo transporta y nunca vuelve a falso.

4. Costo: Se recalcula sumando el costo del terreno de cada casilla destino (sección 7) y debe coincidir con el costo que reporta el algoritmo.

5. Meta: El último estado es prueba.objetivo o en la misión llevar_recurso, la base del bando de la unidad con lleva_recurso verdadero.

6. Sin solución: si el algoritmo reporta que no hay camino el resultado debe tener el camino vacío y costo 0.

Partidas: verificar_partida recorre de nuevo una partida completa (modo partida, IA contra IA con Minimax o alfa-beta) desde el estado inicial del JSON. Usa las reglas del juego para saber qué jugadas eran legales y cuál es su efecto (recogida e intercepción), y además comprueba por

su cuenta los invariantes de la sección 9 de modo que un error en las reglas también se detecta. En cada turno revisa: que la jugada era legal y que el estado siguiente es el que producen las reglas tambien que solo cambió la posición de la unidad que se movió, a una casilla vecina (o ninguna, si la jugada es pasar), que esa unidad no quedó en una celda no transitable, en la base rival ni sobre otra unidad, que exactamente uno de recurso y portador tiene valor, que el turno alterna entre A y B y que turnos_jugados aumenta en 1. Al final comprueba que ningún estado intermedio es terminal, que el último sí lo es y que el ganador declarado (o el empate) coincide con el que resulta del estado final.

Escenarios iniciales: Las comprobaciones de casilla ocupada, base rival y celda no transitable se aplican solo a la unidad que se mueve porque la regla 8 acepta escenarios iniciales con dos unidades en la misma casilla o una unidad sobre la base rival por lo tanto revisar todo el tablero marcaría como inválidas partidas que son legales.

Lo que no comprueba: El verificador decide si una respuesta es válida y si su costo es real pero no si es óptima. Comprobar la optimalidad exigiría resolver el problema otra vez y esa comparación se hace en los experimentos (sección 16) por ejemplo confirmando que A* y UCS obtienen el mismo costo.

Uso: El verificador se ejecuta en las pruebas sobre todos los algoritmos y escenarios, en la interfaz después de cada búsqueda o partida para mostrar si el resultado está verificado y en los experimentos antes de registrar cada resultado. Las pruebas también le entregan respuestas alteradas a propósito (costo cambiado, salto de dos casillas, paso sobre un muro, acción que no coincide con el movimiento, meta no alcanzada, recurso que aparece sin pasar por él, jugada ilegal, unidad movida a una casilla ocupada, ganador falso) y verifican que cada defecto se detecta.

16. Metodología experimental

Los experimentos se ejecutan con un único script, experiments/ejecutar_experimentos.py, que carga cada escenario con el mismo cargador de la aplicación, ejecuta los algoritmos con los parámetros de cada caso, verifica cada resultado con el verificador de la sección 15 y guarda las tablas en formato CSV y las gráficas en experiments/resultados. Así cualquier resultado del

informe se reproduce con un solo comando: .venv\Scripts\python experiments\ejecutar_experimentos.py. Ningún resultado se registra si su verificación falla.
Métricas de búsqueda: algoritmo, heurística y k cuando aplican, éxito, costo del camino, número de movimientos, estados generados, estados expandidos, tamaño máximo de la frontera y tiempo en milisegundos. Las métricas de conteo son deterministas porque los algoritmos usan el mismo orden de generación y las mismas reglas de conteo; el tiempo se mide con time.perf_counter, se repite cinco veces cada ejecución y se reporta la mediana para reducir el ruido del sistema.
Métricas adversariales: acción seleccionada, valor Minimax, profundidad, nodos generados, nodos evaluados, nodos podados y tiempo, que son las métricas que pide el enunciado. En las partidas completas se registran además el ganador o el empate y el número de turnos.
Parámetros: orden de direcciones por defecto arriba, derecha, abajo, izquierda, salvo en los experimentos de orden, donde se prueban las 24 permutaciones; k = 1, 2, 4 y 8 para Beam Search; profundidades 1 a 5 para Minimax y alfa-beta; pesos de la utilidad de la sección 10 (100, 5, 2 y 3).

Casos mínimos: Ccada caso del enunciado se asocia a un escenario y a un resultado esperado, obtenido al diseñar el escenario y confirmado por los experimentos:
Caso 1, BFS frente a DFS: escenario del enunciado (scenarios/dev/enunciado_4x4.json). Ambos encuentran solución con caminos y número de expansiones distintos: BFS llega en 4 movimientos con costo 14 y DFS en 6 movimientos con costo 10. Se repite DFS con otro orden de direcciones para mostrar que su camino depende del orden.
Caso 2, ruta corta frente a ruta económica: mismo escenario. BFS devuelve el camino de 4 movimientos y costo 14 que atraviesa bosque y pantano; UCS devuelve el de 6 movimientos y costo 7 que rodea por la columna 0.
Caso 3, obstáculo y heurística: scenarios/casos/caso3_muro.json, mapa de 5 × 7 con un muro vertical entre la unidad y el objetivo. El objetivo está a 4 casillas en línea recta, pero el camino real cuesta 10 porque obliga a rodear el muro.
Caso 4, comparación de heurísticas: mismo escenario con A* usando h1 (Manhattan) y h2

(pasos rodeando obstáculos). Ambas encuentran el costo óptimo de 10; h2 expande menos estados porque no considera prometedoras las casillas pegadas al muro.

Caso 5, Beam Search: misión llevar_recurso sobre el mapa del enunciado (scenarios/casos/caso6_mision.json) con k = 1, 2, 4 y 8, comparando con A*. Se espera que k = 1 no encuentre solución, que k = 2 y k = 4 devuelvan caminos más caros que el óptimo y que k = 8 alcance el costo 15 de A*.

Caso 6, estado compuesto: scenarios/casos/caso6_mision.json y scenarios/casos/caso6_mision_grande.json. En la misión, el camino pasa dos veces por la misma casilla, una sin el recurso y otra con él; en el mapa grande el recurso está en un callejón con una sola entrada, de modo que sin lleva_recurso en el estado la misión no tendría solución. Con UCS el costo es 15 en el mapa del enunciado y 21 en el mapa grande.

Caso 7, decisión adversarial: estado inicial de scenarios/casos/caso7_8_10_adversarial.json (el mapa del enunciado en modo adversarial) con MAX = A. Con profundidad 1 y 2 la jugada recomendada es A2 abajo, que avanza hacia el recurso; con profundidad 3 es A1 derecha, que no se acerca al recurso sino que prepara la intercepción del portador rival.

Caso 8, Minimax frente a alfa-beta: profundidades 1 a 5 sobre caso7_8_10_adversarial.json y sobre scenarios/casos/caso9_orden.json (el mapa grande en modo adversarial). Ambos algoritmos deben seleccionar la misma acción con el mismo valor; alfa-beta debe generar menos nodos. En el escenario del enunciado, con profundidad 4, Minimax genera 1136 nodos y alfa-beta 234.

Caso 9, orden de acciones: alfa-beta con profundidad 5 sobre caso9_orden.json con las 24 permutaciones del orden de direcciones. El valor de la decisión no cambia se compara el número de nodos podados y generados para cada orden.

Caso 10, profundidad: el estado inicial de caso7_8_10_adversarial.json se evalúa con profundidades 1 a 5 con Minimax y alfa-beta, registrando nodos, tiempo, acción y valor, para mostrar el crecimiento exponencial del árbol y el cambio de decisión entre profundidad 2 y 3.

Experimentos complementarios: partidas completas IA contra IA en el escenario del enunciado y en el mapa grande con profundidades 2, 3 y 4, registrando ganador o empate y número de turnos; y sensibilidad al costo del terreno, repitiendo UCS y A* tras cambiar en el JSON el costo

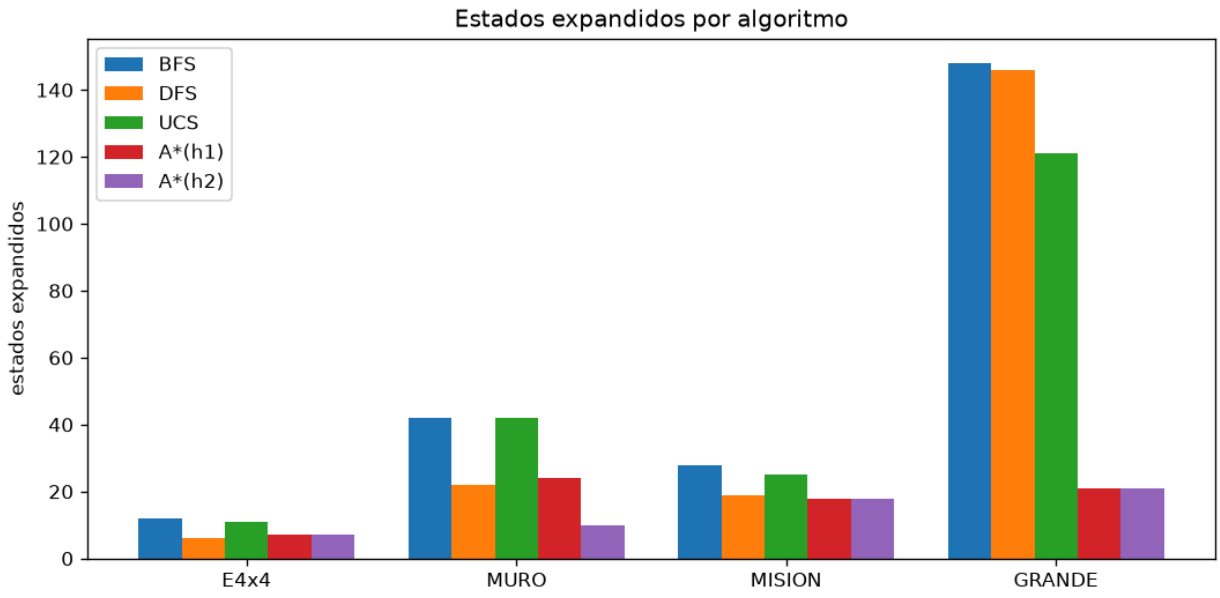
del pantano de 7 a 15, como propone el enunciado, para mostrar que los resultados dependen de los costos del archivo y no del código.

Entorno: Python 3.11 o superior, sin dependencias en tiempo de ejecución; pandas y matplotlib solo se usan para generar tablas y gráficas. Se registra en cada CSV la versión de Python y el sistema operativo, porque el tiempo depende de la máquina, mientras que las demás métricas no.

17. Resultados y análisis

Los resultados de esta sección se obtuvieron con experiments/ejecutar_experimentos.py (sección 16) en Python 3.12.3 sobre Windows 10 y están completos en los archivos CSV de experiments/resultados. Las 50 comprobaciones de resultado esperado del archivo resumen.txt se cumplieron y todos los resultados pasaron el verificador de la sección 15. Las métricas de conteo fueron idénticas en las cinco repeticiones los tiempos que se citan son medianas y dependen de la máquina.

Tabla 1, búsqueda (costo / estados expandidos):



[Figura 1 expandidos_busqueda]

Caso 1, BFS frente a DFS: Ambos encuentran solución en todos los escenarios pero recorren el espacio de forma distinta. En el mapa del enunciado BFS expande 12 estados para garantizar el camino de menos movimientos (4) mientras DFS expande solo 6 porque avanza en profundidad sin retroceder y devuelve un camino de 6 movimientos y costo 10. El camino de DFS depende del orden de generación: con las 24 órdenes posibles su costo varía entre 8 y 35 y con el orden abajo, izquierda, arriba, derecha es 16. En el mapa grande la diferencia es extrema: DFS devuelve un camino de 99 movimientos y costo 246 frente a 21 de BFS. DFS trabaja menos en algunos casos pero no ofrece ninguna garantía sobre la calidad del camino.

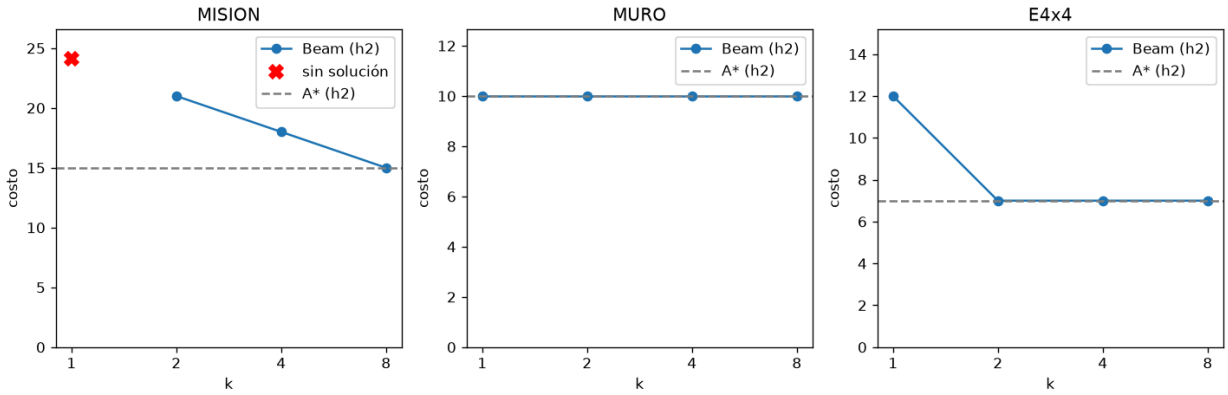
Caso 2, ruta corta frente a ruta económica: En el mapa del enunciado BFS devuelve el camino de 4 movimientos y costo 14 que atraviesa bosque (4) y pantano (7) mientras UCS devuelve el de 6 movimientos y costo 7 que rodea por la columna 0. BFS minimiza movimientos y UCS minimiza costo, cuando los terrenos tienen costos distintos las dos respuestas difieren.

Caso 3, obstáculo y heurística: En el mapa del muro el objetivo está a 4 casillas en línea recta pero el camino óptimo cuesta 10 porque el muro obliga a rodearlo por la fila 4. Sin heurística, BFS y UCS expanden 42 estados, la diferencia entre la estimación en línea recta y el costo real es la que pone a prueba las heurísticas.

Caso 4, comparación de heurísticas: En el mismo escenario A* encuentra el costo óptimo de 10 con ambas heurísticas, pero h1 (Manhattan) expande 24 estados y h2 (pasos rodeando obstáculos) expande 10, exactamente los del camino. h1 considera prometedoras las casillas pegadas al muro y las explora antes de rodearlo pero h2 sabe cuántos pasos faltan realmente. En el mapa grande ambas heurísticas expanden 21 estados frente a 121 de UCS: con el mismo costo óptimo, la heurística reduce la exploración casi seis veces.

Caso 5, Beam Search: En la misión, con h2, k = 1 no encuentra solución, k = 2 devuelve costo 21, k = 4 costo 18 y k = 8 el óptimo de 15, igual que A*. Con k = 1 Beam rodea el recurso sin entrar porque la casilla de pantano siempre tiene la mayor f y se descarta, y termina sin estados

pendientes. Al aumentar k conserva más alternativas y cuando k alcanza el tamaño máximo de la frontera de A* (6), se comporta igual que A*. La calidad de la heurística también influye: en el mapa del muro, Beam con h2 obtiene el óptimo ya con k = 1, mientras con h1 necesita k = 4 (con k = 1 y 2 obtiene costo 16). En el mapa del enunciado, k = 1 devuelve costo 12 y k ≥ 2 el óptimo de 7.



[Figura 2: beam_k]

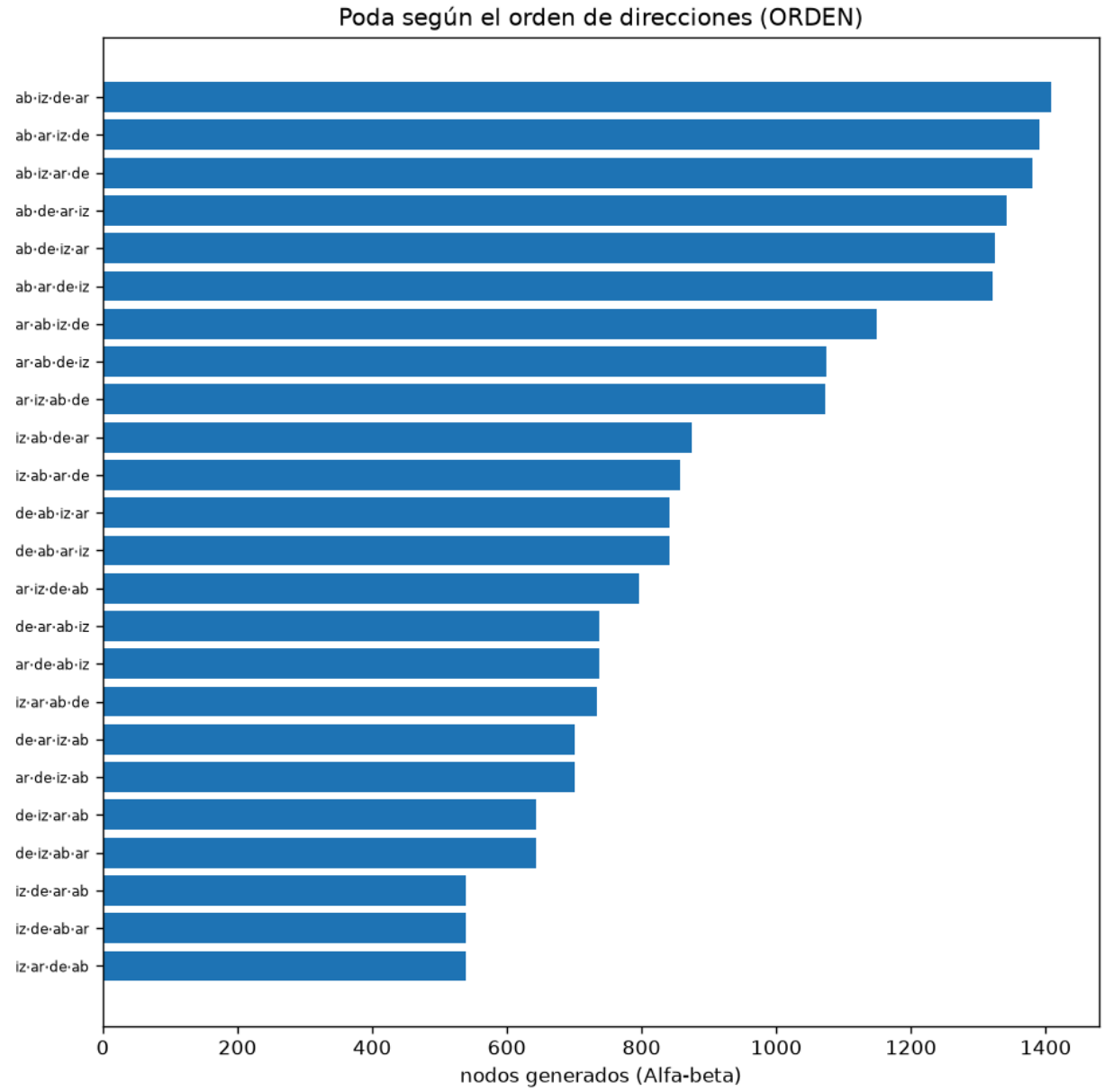
Caso 6, estado compuesto: En la misión del mapa del enunciado UCS obtiene costo 15 con un camino que pasa dos veces por (1,0), (2,0) y (2,1) una sin el recurso y otra con él. En el mapa grande obtiene costo 21 y sale del callejón del recurso por la misma casilla (3,6) por la que entró. En ambos casos la solución existe solo porque (fila, columna, falso) y (fila, columna, verdadero) son estados distintos y con la posición sola, la casilla de entrada quedaría marcada como visitada y la búsqueda no podría volver.

Caso 7, decisión adversarial: En el estado inicial del enunciado, con profundidad 1 y 2 la acción recomendada es A2 abajo que acerca a A al recurso. Con profundidad 3 y superiores es A1 derecha: Minimax ya ve que B llega primero al recurso y prefiere colocar a A1 donde podrá interceptar al portador en lugar de avanzar directamente hacia el recurso.

Caso 8, Minimax frente a alfa-beta: En las 58 comparaciones realizadas (profundidades 1 a 5 y 24 órdenes) alfa-beta seleccionó la misma acción con el mismo valor que Minimax. En el escenario del enunciado con profundidad 5 Minimax generó 5947 nodos y alfa-beta 842 (180

podas) en el mapa grande 11 372 frente a 738, es decir unas 15 veces menos con un tiempo de 503 ms frente a 30 ms.

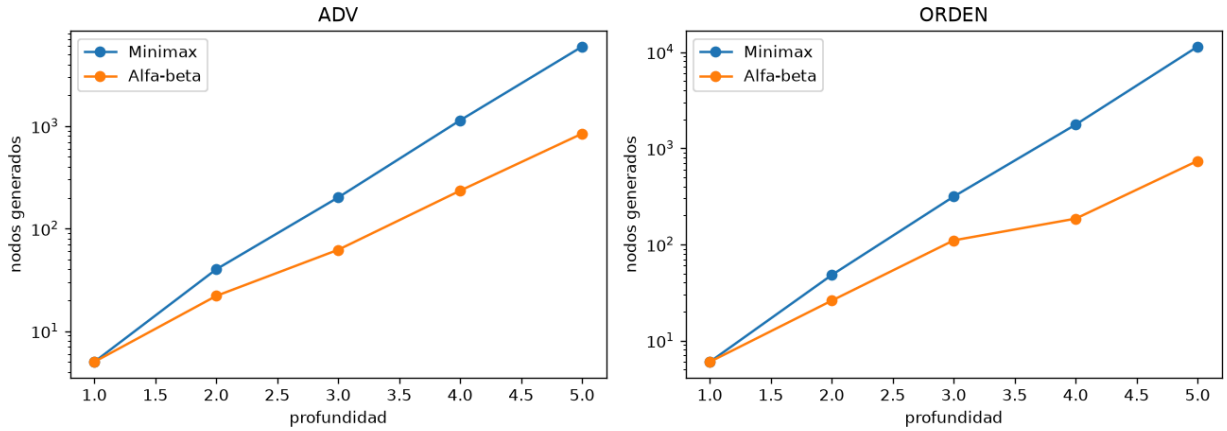
Caso 9, orden de acciones: En el mapa grande con profundidad 5 alfa-beta generó entre 539 y 1409 nodos según el orden de direcciones siempre con la acción A1 derecha y el valor −6 en el escenario del enunciado con profundidad 4 entre 127 y 343 nodos siempre con A1 derecha y −111. Los órdenes que empiezan por izquierda fueron los más eficientes en el mapa grande y los que empiezan por abajo los menos eficientes. Una explicación consistente con los datos es que las mejores respuestas de B cuyas unidades están a la derecha del recurso son moverse a la izquierda cuando esas jugadas se generan primero, beta se ajusta antes y se cortan más ramas. El número de podas por sí solo no mide el ahorro (un mal orden puede producir más cortes, pero más tardíos) la medida relevante son los nodos generados.



[Figura 3: orden_poda]

Caso 10, profundidad: Con Minimax los nodos generados crecen de forma exponencial con la profundidad: 5, 40, 201, 1136 y 5947 en el escenario del enunciado (unas cinco veces por nivel) y 6, 48, 314, 1758 y 11 372 en el mapa grande (unas seis veces por nivel). Alfa-beta crece mucho más despacio, de 5 a 842 y de 6 a 738 nodos. La acción recomendada cambia entre la profundidad 2 y la 3 y el valor cambia en cada nivel (−2, −114, 0, −111 y −91) porque cada nivel

agrega una respuesta posible del rival que la evaluación anterior no veía.



[Figura 4: nodos_profundidad]

Partidas completas: En el escenario del enunciado con profundidad 2 gana B en 18 turnos y con profundidades 3 y 4 la partida termina en empate por límite de turnos. En el mapa grande las tres profundidades terminan en empate: B llega primero al recurso, que está en un callejón con una sola salida y A se coloca frente a esa salida, si B sale es interceptado y si A se acerca B escapa de modo que ninguno puede mejorar su posición y la regla del límite de turnos cierra la partida. Una partida completa con profundidad 4 en el mapa grande tardó 3,2 segundos.

Sensibilidad al costo del terreno: Al cambiar en el JSON el costo del pantano de 7 a 15 la ruta óptima del enunciado no cambia (costo 7) porque no pasa por el pantano mientras el costo de la misión sube de 15 a 23 con el mismo camino, porque el recurso está sobre el pantano y pisarlo es obligatorio. Los algoritmos toman los costos del archivo y no de constantes en el código.

Respuesta a la pregunta central: Al buscar solo una ruta BFS y DFS encuentran soluciones sin considerar su costo y se diferencian únicamente en cómo exploran. Al optimizar el costo UCS garantiza la ruta más barata a cambio de explorar más. Con conocimiento heurístico A* mantiene esa garantía y reduce la exploración hasta seis veces, tanto más cuanto mejor es la heurística y Beam Search muestra el compromiso entre memoria y calidad de la solución. Frente a un adversario, la decisión deja de ser el mejor camino y pasa a ser la mejor acción frente a las

respuestas posibles del rival: con más profundidad el sistema anticipa amenazas como la intercepción y cambia su decisión pero el costo crece de forma exponencial. alfa-beta conserva la misma decisión explorando una fracción del árbol y el orden de las jugadas determina cuánto ahorra. Al mismo tiempo, el límite de profundidad explica los bloqueos que terminan en empate: el sistema decide bien hasta donde alcanza a ver.

18. Análisis de complejidad

Notación: N es el número de celdas transitables V el número de estados de navegación a lo sumo 2N porque el estado agrega lleva_recurso b el factor de ramificación, que en navegación es como máximo 4 d la profundidad de la solución más superficial; m la profundidad máxima del espacio L el largo de un camino u el número de unidades por bando T el número de turnos de una partida; y en el juego $b \leq 4u + 1$.

BFS: Tiempo O(b^d) en la formulación de árbol como la búsqueda es en grafo y cada estado se encola una sola vez, queda acotado por O(V). Espacio O(V) por la frontera y el conjunto de alcanzados. Es completo y óptimo en número de movimientos, no en costo.

DFS: Tiempo O(b^m) en árbol, acotado por O(V) en grafo porque cada estado se expande una vez. Espacio O(V) por el conjunto de visitados la frontera crece como $O(b \cdot m)$ en el peor caso. Es completo en espacios finitos con control de repetidos, pero no es óptimo.

UCS: En grafo, cada estado se expande una vez y cada expansión inserta a lo sumo 4 entradas en el montículo, por lo que el tiempo es O(V log V) y el espacio O(V). En la formulación de árbol el costo es $O(b^{(1 + C^*/\varepsilon)})$ con C^* el costo óptimo y ε el menor costo de terreno. Es óptimo porque todos los costos son positivos.

A*: El peor caso es el mismo de UCS, O(V log V) en tiempo y O(V) en espacio pero con una heurística consistente solo expande estados con f menor o igual al costo óptimo, lo que reduce las expansiones en la práctica. h1 se calcula en O(1); h2 requiere un BFS previo sobre las celdas O(N), una vez por búsqueda (dos en la misión) y luego cada consulta es O(1).

Beam Search: Cada expansión inserta a lo sumo 4 sucesores y si la frontera supera k la recorta ordenando k + 4 elementos, con costo O(k log k). Con a lo sumo V expansiones, el tiempo es $O(V \cdot k \log k)$. La frontera ocupa O(k) el conjunto de expandidos y los padres ocupan O(V). Con

k pequeño la memoria de la frontera es mínima a cambio de perder completitud y optimalidad.
Minimax: Explora el árbol completo hasta la profundidad p: tiempo O(b^p) nodos con $b \leq 4u + 1$. Cada nodo cuesta O(u) para generar jugadas, aplicar una jugada y evaluar la utilidad que solo consulta la tabla de distancias. Espacio $O(b \cdot p)$ por la recursión y las listas de jugadas de cada nivel.
Alfa-beta: Mismo resultado que Minimax con tiempo O(b^(p/2)) en el mejor caso cuando las mejores jugadas se generan primero y O(b^p) en el peor caso, por eso el orden de generación afecta la cantidad de poda (caso 9). Espacio $O(b \cdot p)$. En el escenario del enunciado con profundidad 4 Minimax genera 1136 nodos y alfa-beta 234.
Tabla de distancias: Un Dijkstra desde cada celda transitable O(N log N) cada uno para un total de $O(N^2 \log N)$ en tiempo y $O(N^2)$ en espacio. En un mapa de 20×20 son unas 400 ejecuciones y 160 000 distancias, calculadas una sola vez al crear el evaluador.
Partida completa: T decisiones cada una con el costo de alfa-beta a la profundidad elegida; T está acotado por el límite de turnos (200 por defecto).
Carga, validación y verificación: La carga y la validación son lineales en el tamaño del JSON, $O(F \cdot C + U)$ para un mapa de F filas, C columnas y U unidades. verificar_camino es O(L). verificar_partida es $O(T \cdot u)$, porque en cada turno vuelve a generar las jugadas legales y a aplicar la jugada.

19. Supuestos y limitaciones

Supuestos y limitaciones del modelo, cada uno elegido a propósito para mantener el problema claro y defendible:

- No hay combate ni eliminación de unidades; interceptar solo cambia quién lleva el recurso.
- Todas las unidades son idénticas; el campo tipo del JSON se conserva pero no altera el comportamiento.
- El movimiento es ortogonal, de una casilla por jugada, sin diagonales; en cada turno se mueve una sola unidad y no hay acciones simultáneas.
- Una vez recogido, el recurso nunca vuelve al suelo, porque la intercepción lo transfiere

directamente. Por eso la carrera por el recurso solo ocurre al inicio de la partida.

- La navegación ignora a las demás unidades, así que un camino de navegación puede estar bloqueado durante una partida. Las distancias que usa la función de utilidad también ignoran las unidades: un bloqueo no se refleja en la evaluación, solo en la exploración de Minimax.
- La función de utilidad es una estimación con pesos iniciales, no garantiza juego óptimo. Como Minimax explora con profundidad limitada, puede no ver consecuencias que ocurren después de esa profundidad.
- El límite de turnos es un valor convencional (200 por defecto) y configurable.
- Los campos prueba.mision y juego.limite_turnos son extensiones propias y opcionales del formato común; un escenario que no los incluya se comporta según los valores por defecto.

20. Reporte de uso de IA

El equipo usó IA generativa como herramienta de apoyo durante todo el proyecto como permite el enunciado. Esta sección declara en qué partes se usó, cómo se usó y cómo se validó cada aporte. La trazabilidad a nivel de código está en el docstring de cada método público que registra si se usó IA (AI usage), qué hizo (AI intervention).

Herramientas: Claude (Anthropic) en dos modalidades: un asistente conversacional en claude.ai usado para analizar el enunciado, discutir el modelado, revisar planes, simular resultados de referencia y Claude Code en el repositorio, usado para generar el código a partir de especificaciones escritas por paso.

Modelado y decisiones de diseño (secciones 1 a 12): El asistente explicó cada punto del enunciado y para cada decisión abierta, presentó alternativas con sus ventajas y desventajas. Las decisiones las tomó el equipo: estado de navegación con lleva_recurso, unidades idénticas, costo de la celda destino, límite de turnos configurable con 200 por defecto, ocupación de casillas, prohibición de entrar a la base rival, intercepción sin eliminación, MAX como el bando con el turno en la raíz, pesos de la función de utilidad, segunda heurística de pasos rodeando obstáculos

y criterio f = g + h para Beam Search.

Desarrollo del código: El trabajo se dividió en diez pasos (carga, validación, problema de navegación, búsqueda no informada, búsqueda informada, reglas del juego, utilidad y Minimax, verificación y partida, interfaz y experimentos). En cada paso el flujo fue: (1) el equipo definía el alcance con el asistente, que redactaba una especificación con las reglas exactas y los resultados esperados (2) Claude Code generaba un plan de implementación (3) el equipo y el asistente revisaban el plan antes de aprobarlo y pedían correcciones cuando no coincidía con el modelado (4) Claude Code implementaba el código y sus pruebas (5) el equipo ejecutaba pytest y ruff, probaba manualmente y hacía el commit e incorporación al repositorio.

Validación: Cada aporte se validó por varias vías independientes: pruebas automáticas con pytest para cada módulo con valores esperados calculados antes de implementar, simulaciones de referencia escritas aparte por el asistente cuyos resultados coincidieron con los del código el verificador independiente de la sección 15 que revisa cada camino y cada partida sin depender de los algoritmos, pruebas manuales en la interfaz con los escenarios de cada caso mínimo y el script de experimentos, cuyas 50 comprobaciones de resultado esperado se cumplieron.

Interfaz y experimentos: La interfaz en Tkinter y el script de experimentos también se generaron con Claude Code a partir de especificaciones del equipo donde se decidió su alcance. Los escenarios de los casos mínimos se diseñaron con el asistente y se comprobaron en la interfaz antes de usarlos en los experimentos. El análisis de resultados de la sección 17 lo redactó el asistente a partir de los archivos CSV generados por el equipo y el equipo lo revisó.

Revisión por integrante: El equipo revisó el código generado en cada paso. La revisión consistió en: leer el plan de implementación antes de aprobarlo y comprobar que coincidiera con las decisiones de modelado del documento, leer el código generado para entender su funcionamiento y confirmar que tuviera sentido frente a las reglas definidas, ejecutar las pruebas automáticas (pytest) y el análisis estático (ruff) después de cada paso comprobando que los valores esperados coincidieran con los obtenidos y probar manualmente en la interfaz los escenarios de los casos mínimos.

Responsabilidad: El uso de IA no reemplazó la comprensión del problema: cada algoritmo, regla y resultado se discutió antes de aceptarse.